



3

17

2040 SP

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Geometrical Transformations Viewing



د. عمار النحاس

محتوى مجاني غير مخصص للبيع التجاري
21/10/2024

بيانيات حاسوبية

تحدثنا في المحاضرة السابقة عن جزء من الجرافيكس وهو الـ Modeling وسنتحدث في هذه المحاضرة عن الـ Rendering لكن قبل البدء علينا مراجعة بعض المفاهيم الرياضية الهامة لفهم المحاضرة ولفهم التطبيق في العملي.

Mathematical Recall

1. المصفوفات (Matrix):

تعرفنا سابقاً في مادة الجبر الخطي على المصفوفات والعمليات عليها وسنتذكر بعض الأفكار الهامة:

- لجمع مصفوفتين يجب ان يكون لهما نفس الابعاد (dimensions) أي عدد أسطر الأولى يساوي عدد أسطر الثانية وعدد أعمدة الأولى يساوي عدد أعمدة الثانية.
- نقوم بجمع كل عنصر من المصفوفة الأولى مع العنصر الموافق له من المصفوفة الثانية.
- إن جمع المصفوفات تبديلي (commutative): $a + b = b + a$
- جمع المصفوفات تجميعي (associative): $(a + b) + c = a + (b + c)$
- يمكن ضرب المصفوفات بعدد scalar
- لنفي مصفوفة نضرب كل عنصر من عناصرها بـ -1 أي: $-A = -1 \cdot A$
- يمكن كتابة $A - B$ بالشكل $A + (-B)$ علماً أن A و B لها نفس عدد الأسطر ونفس عدد الأعمدة.
- طرح المصفوفات غير تبديلي (Not Commutative).
- لضرب مصفوفتين يجب أن يكون عدد أعمدة الأولى يساوي عدد أسطر الثانية:

مثال:

$$\begin{bmatrix} 2 & 0 & 3 \\ 1 & -4 & -2 \end{bmatrix} \cdot \begin{bmatrix} 3 & 1 & 0 \\ -1 & 5 & 2 \\ 4 & -1 & 1 \end{bmatrix} = \begin{bmatrix} 18 & -1 & 3 \\ -1 & -17 & -10 \end{bmatrix}$$

حيث: $-1 = (2 \times 1) + (0 \times 5) + (3 \times -1)$, $18 = (2 \times 3) + (0 \times -1) + (3 \times 4)$ نضرب عناصر السطر الأول مع أعمدة المصفوفة الثانية وهكذا

- إن ضرب المصفوفات غير تبديلي (Not Commutative) $A \cdot B \neq B \cdot A$
- ضرب المصفوفات تجميعي (associative) $(AB)C = A(BC)$
- ضرب المصفوفات توزيعي على الجمع $A(B + C) = AB + AC$ (distributive over matrix adding)
- منقول مصفوفة (The transpose of a matrix) هو مصفوفة جديدة أسطرها هي أعمدة المصفوفة الأصلية وأعمدتها هي أسطر المصفوفة الأصلية:

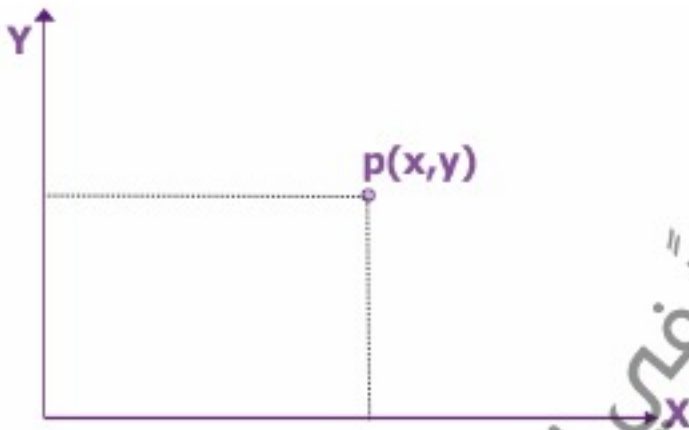
مثال:

$$A = \begin{bmatrix} 5 & 2 & 4 \\ 3 & 4 & 6 \\ 8 & 1 & 5 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 5 & 3 & 8 \\ 2 & 4 & 1 \\ 4 & 6 & 5 \end{bmatrix}$$

2. الإحداثيات الديكارتية في المستوي (Cartesian coordinate in the plane):

أي نقطة p في المستوي تحدد بـ عددين حقيقيين يشير كل منهما إلى موقع المسقط العامودي من النقطة على خطوط ثابتة ومتعامدة تسمى المحاور $axes$.
مبدأ الاحداثيات هو النقطة المركزية والمرجعية لجميع النقاط والذي تتقاطع فيه المحاور.
النقطة (point): هي موقع في الفضاء لا تمتلك أي حجم أو وحدة قياس .
النقطة في الكمبيوتر غرافيكس غالباً تكون رأس في الأشكال 3D



3. الأشعة (Vectors)

- للشعاع جهة (direction) وطويلة (length).
- نستخدم مصفوفة أحادية البعد عامودية لتمثيل شعاع (أو نقطة) مثلاً:

$$\vec{u} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

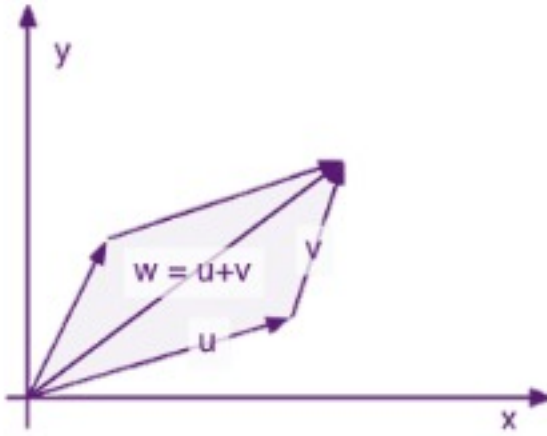
- جمع وطرح الشعاع:
- لدينا الشعاعان u, v :

$$u = (u_1, u_2, u_3) = u_1i + u_2j + u_3k$$

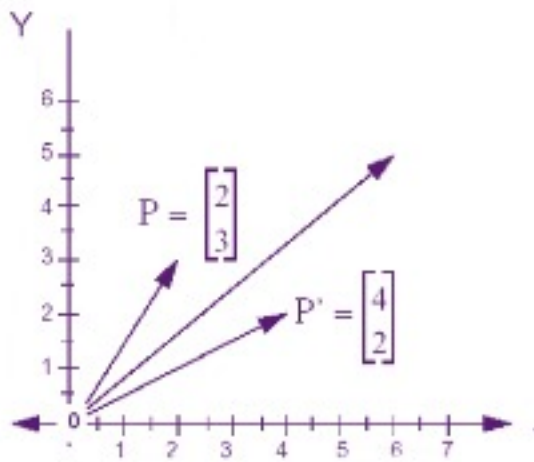
$$v = (v_1, v_2, v_3) = v_1i + v_2j + v_3k$$



$$\begin{aligned} u \pm v &= (u_1 i, u_2 j, u_3 k) \pm (v_1 i, v_2 j, v_3 k) \\ &= (u_1 \pm v_1) i + (u_2 \pm v_2) j + (u_3 \pm v_3) k \\ &= (u_1 \pm v_1, u_2 \pm v_2, u_3 \pm v_3) \end{aligned}$$



يمكن أيضاً الجمع بعد التمثيل بمصفوفة:



$$P + P' = \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 4 \\ 2 \end{bmatrix} = \begin{bmatrix} 6 \\ 5 \end{bmatrix}$$

يمكن جمع نقطة مع شعاع وسنرى أن هذا يمثل انسحاب النقطة translation

يمكن ضرب شعاع بقيمة عددية (Scalar Multiplication):

cu is a vector

$$|cu| = |c| |u|$$

$$\text{dir}(cu) = \text{sgn}(c) \text{dir}(u)$$

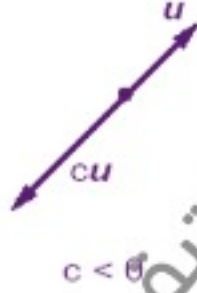
$$= (u_1, u_2, u_3) = u_1 i + u_2 j + u_3 k$$

$$cu = c(u_1 i + u_2 j + u_3 k)$$

$$= (c \cdot u_1) i + (c \cdot u_2) j + (c \cdot u_3) k \\ = (cu_1, cu_2, cu_3)$$

وإذا مثلنا الشعاع كمصفوفة:

$$a \cdot u = a \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax \\ ay \end{bmatrix}$$



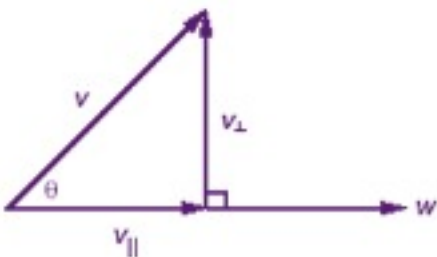
4. الجداء الداخلي لشعاعين (scalar product, or inner product/Dot Product)

يكتب:

$$u \cdot v = |u| \cdot |v| \cos \theta$$

ناتج الجداء الداخلي هو عدد:

u, v is a scalar



استخدامات الجداء الداخلي:

- إيجاد الطويلة: $|v|^2 = v \cdot v$
- إيجاد الزاوية بين شعاعين: $\cos \theta = \frac{w \cdot v}{|w||v|}$
- إثبات التعامد: $v \perp w \iff v \cdot w = 0$

5. اجداء الخارجي لشعاعين (Cross Product):

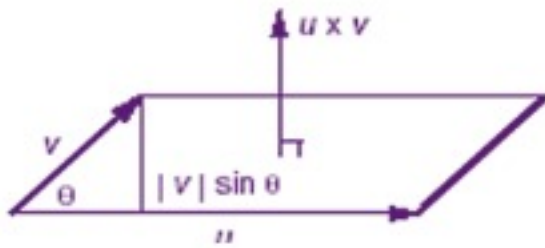
$$|u \times v| = \text{area}(u, v) = |u||v|\sin\theta$$

ناتج الجداء الخارجي هو شعاع: $u \times v$ is vector وهو عامودي على u, v :

$$\text{dir}(u \times v) \perp u, v$$

$$\text{sgn}(u, v, u \times v) = +1$$

ويتم تحديد جهة الشعاع الناتج (للاعلى او للأسفل) باستخدام قاعدة اليد اليمنى (Right hand rule)



$$\text{area}(u, v) = |u \times v|$$

$$(u \times v) \cdot u = 0$$

$$(u \times v) \cdot v = 0$$

$$(u \times v) = 0 \text{ إذا كان } u, v \text{ متوازيان}$$

$$u \times u = 0$$

$$u \times v = -v \times u \text{ (غير تبديلي)}$$

$$u \times (v \times w) = (u \cdot w)v - (u \cdot v)w \text{ (غير توزيعي)}$$

$$|u \times v|^2 = |u|^2 \cdot |v|^2 - (u \cdot v)^2$$

$$u \times (v + w) = u \times v + u \times w \text{ (توزيعي على الجمع)}$$



$$\begin{aligned} u \times v &= (u_1 i + u_2 j + u_3 k) \times (v_1 i + v_2 j + v_3 k) \\ &= (u_2 v_3 - u_3 v_2) i + (u_3 v_1 - u_1 v_3) j + (u_1 v_2 - u_2 v_1) k \\ &= (u_2 v_3 - u_3 v_2, u_3 v_1 - u_1 v_3, u_1 v_2 - u_2 v_1) \\ &= \text{Det} \begin{vmatrix} i & j & k \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix} \end{aligned}$$

Geometrical Transformations

هي أي عملية يتم تطبيقها على غرض (object) بهدف التغيير في خصائصه الهندسية (الحجم الموقع الجهة)

سنتحدث عن أهم ثلاث تحويلات هندسية:

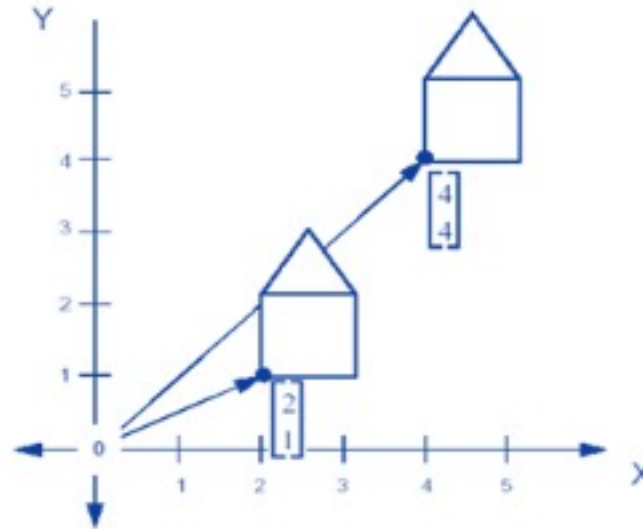
■ Translation الانسحاب

■ Rotation الدوران

■ Scaling التقييس

1. الانسحاب translation

لدينا الشكل الآتي في جملة الإحداثيات ثنائية البعد:



لتغيير موقع الشكل نطبق عليه عملية انسحاب وذلك باستخدام جمع الأشعة:

$$P' = P + T$$

حيث:

$P = \begin{bmatrix} x \\ y \end{bmatrix}$ النقطة التي نطبق عليها الانسحاب

$P' = \begin{bmatrix} x' \\ y' \end{bmatrix}$ النقطة بعد الانسحاب

$T = \begin{bmatrix} dx \\ dy \end{bmatrix}$ حيث dx مقدار الانسحاب على المحور x و dy مقدار الانسحاب على المحور y .

أي قمنا بسحب النقطة وفق الشعاع الانسحاب T الذي يحدد مقدار الانسحاب على كل محور لكل نقطة.

نطبق شعاع الانسحاب على كل نقطة من نقاط الشكل ليتم سحب الشكل بالكامل.

الانسحاب يحافظ على الأطوال وعلى الزوايا.

2. التقييس (Scaling)

نستخدم الضرب لتغيير حجم الشكل (تكبير أو تصغير):

لا يمكن ضرب شعاع بنقطة لذا سنضرب بمصفوفة تسمى مصفوفة التقييس (Scaling Matrix) نرمز لها بـ S وهي

مصفوفة قطرها الرئيسي هو معاملات التقييس على كل من x و y و (S_x, S_y) وقطرها الثانوي أصفار

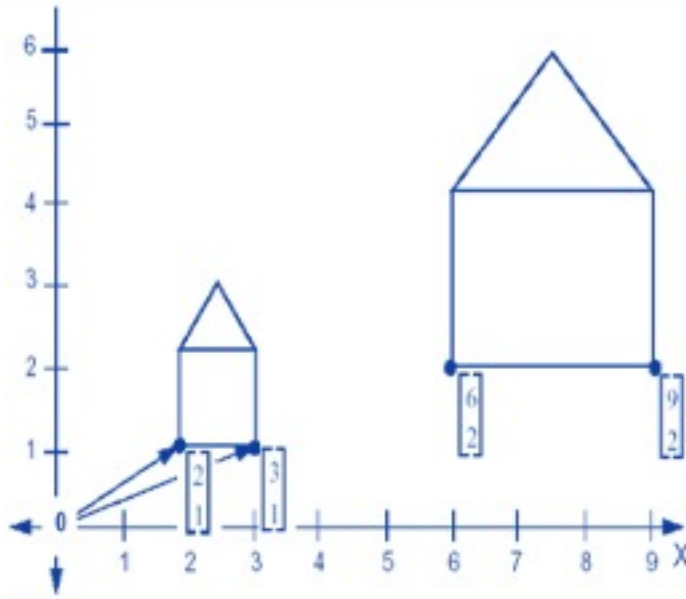
$$P' = SP$$

حيث:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\Rightarrow x' = S_x \cdot x \quad y' = S_y \cdot y$$





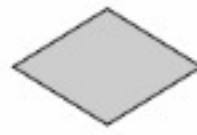
■ شرط الضرب هنا محقق (عدد الأعمدة الأولى يساوي عدد اسطر الثانية)

■ الضرب يكون من اليسار الى اليمين أي نضرب المصفوفة من يسار النقطة والضرب غير تبديلي.

■ التقييس لا يحافظ على الاطوال.

■ التقييس لا يحافظ على الزوايا ($Sx \pm Sy$)

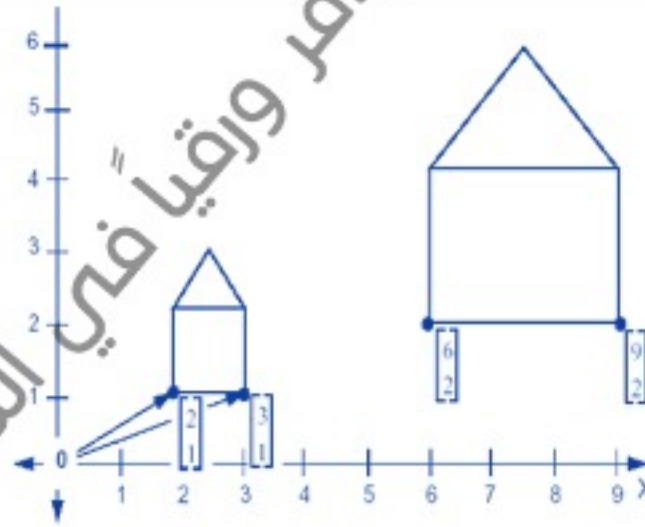
نسمي التقييس في حالة $Sx = Sy$ تقييس منظم uniform scaling



Non-uniformly scaled

ال Scaling يحافظ على التوازي ولا يحافظ على الاطوال والزوايا ونسميه Affine transformation

⚡ مشكلة ال scaling انه يقوم بعمل انزياح (translate) ضمناً:



لحل هذه المشكلة نقوم برسم الجسم بحيث يكون مركزه هو مبدأ الإحداثيات (0,0,0) وبما ان أي عدد ضرب صفر هو صفر لن يتحرك الجسم عن مركزه.
نقوم بعد ذلك بعمل translate للجسم الى الاحداثيات التي نريد.

3. الدوران Rotation

الدوران يكون بزاوية θ وحول نقطة هي مبدأ الإحداثيات

$$P' = R_{\theta} \cdot P \quad \text{حيث} \quad R_{\theta} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}, P = \begin{bmatrix} x \\ y \end{bmatrix}, P' = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

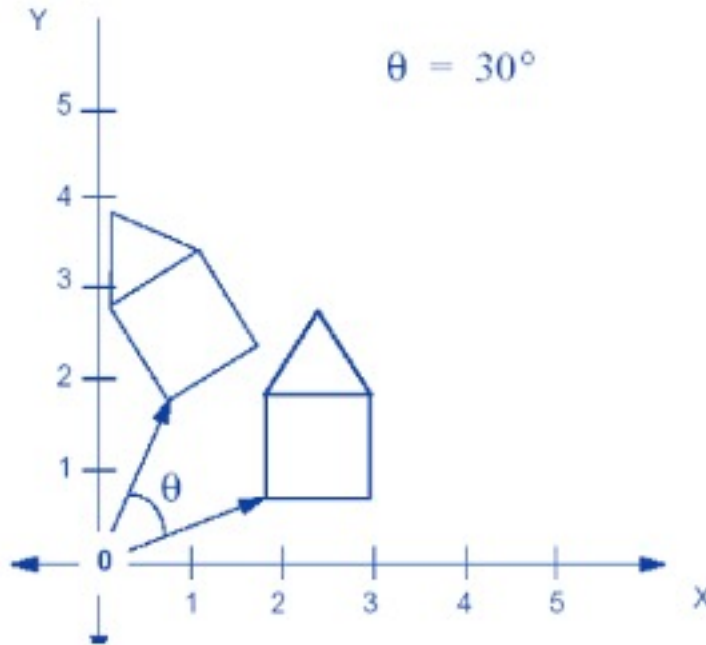
وتكون معادلات الاحداثيات الجديدة:

$$\begin{aligned}x' &= x \cdot \cos\theta - y \cdot \sin\theta \\y' &= x \cdot \sin\theta + y \cdot \cos\theta\end{aligned}$$

نسمي R_θ مصفوفة الدوران

الدوران يحافظ على الأطوال والزوايا

❗ مشكلة الدوران أيضاً انه يطبق انزياح ضمناً:



لحل هذه المشكلة نضع الجسم أيضاً في مبدأ الاحداثيات فيكون مركز الجسم هو مركز الدوران فيبقى في مكانه.

الاحداثيات المتجانسة Homogeneous Coordinate

الانتقال من n بُعد (n dimensional) الى $n+1$ بُعد ($n+1$ dimensional)

$$\begin{bmatrix} x \\ y \end{bmatrix} \rightarrow \begin{bmatrix} x \\ y \\ h \end{bmatrix}$$

■ القيمة الافتراضية للبعد h هي 1 ويمكن إهماله في هذه الحالة.

■ إذا كانت قيمته لا تساوي الواحد $h \neq 1$ فهو يعبر عن scaling مثلاً إذا كانت قيمته 3 يعني أنه تم التكبير على x, y ثلاث مرات.

مثال:

لدينا النقطة $(10, 10, 5)$ في الاحداثيات المتجانسة، ماذا توافق في الاحداثيات العادية؟

نقوم بالقسمة على h وهنا $h = 5$ فنحصل على $(2, 2, 1)$ وهذا يكافئ النقطة $(2, 2)$ في الاحداثيات العادية.

وبالتالي أي نقطة $\begin{bmatrix} x \\ y \end{bmatrix}$ هي ذاتها النقطة $\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

إذا يمكننا باستخدام الاحداثيات المتجانسة التعبير عن الانسحاب كضرب مصفوفات

لدينا مصفوفة الانسحاب وهي مصفوفة واحدة بالشكل:

$$T(dx, dy) = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

مصفوفة التقييس في الاحداثيات المتجانسة:

$$S(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

مصفوفة الدوران:

$$R_\theta = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

إذا أي تحويل هندسي يمكننا التعبير عنه بمصفوفة 3×3 في 2D او مصفوفة 4×4 في 3D

ماذا نستفيد؟

- أصبح لدينا طريقة عامة لتطبيق أي تحويل هندسي وهي الضرب بمصفوفة هذا التحويل
 - استفدنا من خواص ضرب المصفوفات للحصول على مصفوفة واحدة تقوم بأكثر من تحويل هندسي: مثلاً للقيام بعملية التدوير للجسم علينا وضع مركزه في مبدأ الاحداثيات (translation to origin) ثم الدوران (rotation) ثم لإعادة الى الاحداثيات الأساسية (translation back).
- أي لدينا جداء ثلاث مصفوفات

$$\begin{bmatrix} 1 & 0 & x_1 \\ 0 & 1 & y_1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_1 \\ 0 & 1 & -y_1 \\ 0 & 0 & 1 \end{bmatrix} =$$

(نضرب النقطة من يسارها لذا نضع المصفوفات بهذا الترتيب) تذكر أن ضرب المصفوفات غير تبديلي فيتم تطبيق التحويلات عليها بالترتيب : انسحاب المبدأ - دوران --> انسحاب العودة

بما أن ضرب المصفوفات تجميعي سنضرب المصفوفات الثلاث ببعضها ثم نضرب بالنقطة والناتج مصفوفة مربعة 3×3 تقوم بثلاث تحويلات:

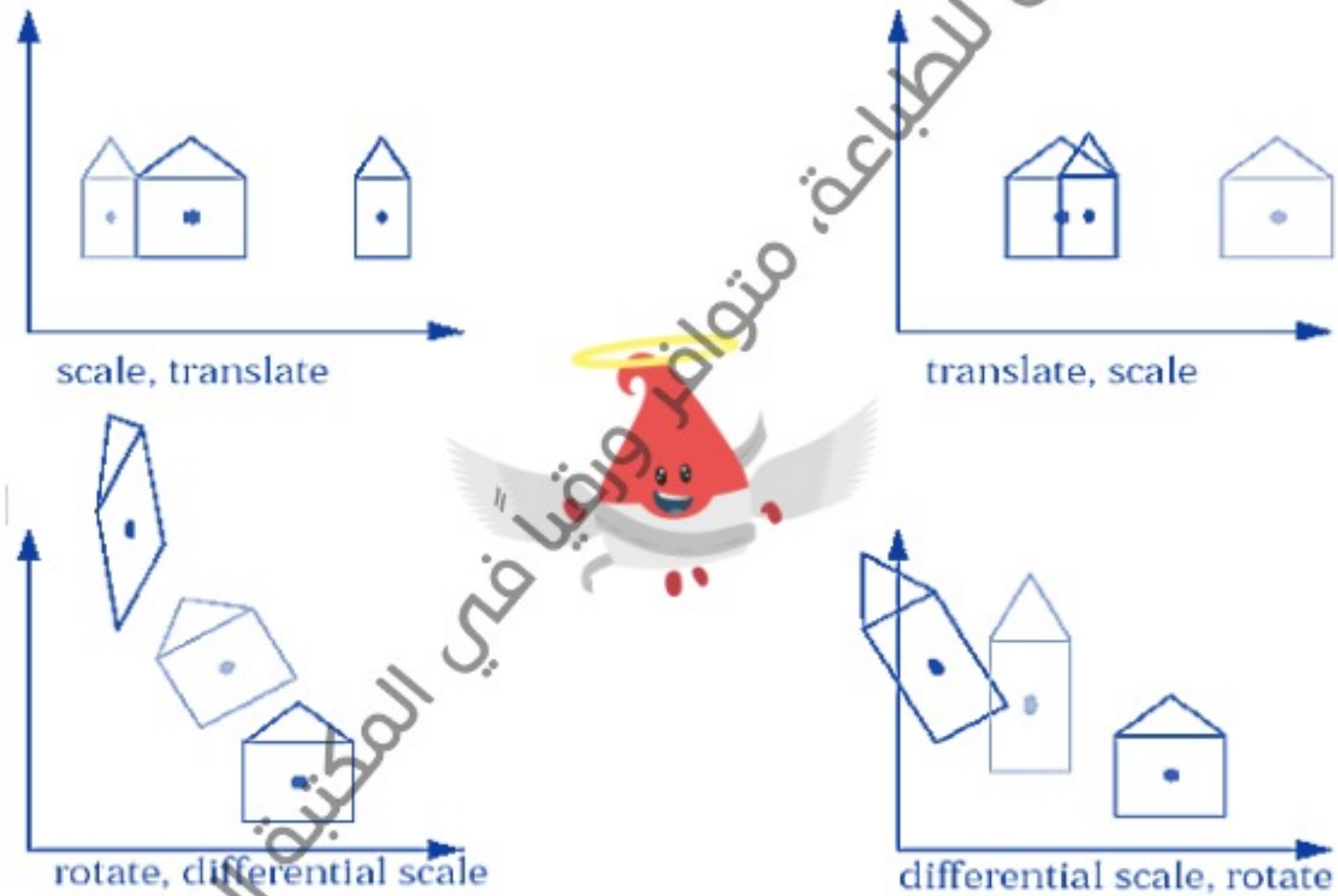
$$\begin{bmatrix} \cos\theta & -\sin\theta & x_1(1 - \cos\theta) + y_1 \sin\theta \\ \sin\theta & \cos\theta & y_1(1 - \cos\theta) - x_1 \sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

تسمى مصفوفة (Model view) وهي توفر سرعة وأداء
في نظام OpenGL يتم تخزين مصفوفة (Model view) وعندما نطبق تحويل هندسي يتم التعديل على هذه المصفوفة
ولكن لسنا مضطرين لنقل الجسم الى مبدأ الاحداثيات وهو حل وحيد لكن الحل المستخدم وهو اعتبار مبدأ الجسم هو
مبدأ الاحداثيات.

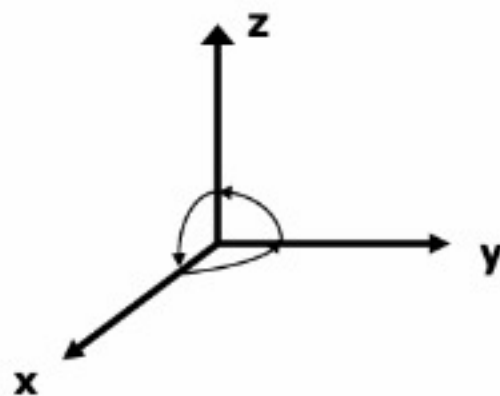
ملاحظة مهمة

إن تطبيق التحويلات الهندسية غير قابل لعكس أي غير تبديلي:

- السبب الرياضي هو ان ضرب المصفوفات في تبديلي
- السبب المنطقي هو انه مثلاً إذا طبقنا انسحاب ثم scale ستقوم عملية ال scale بتطبيق انسحاب ضمناً لذلك نحصل على النتيجة التي نريد:



يمكن تعميم الكلام السابق على جملة احداثيات ثلاثية البعد (3D coordinate system):
تحدد باستخدام قاعدة اليد اليمنى (Right hand):



أحداثيات المتجانسة في 3D رباعية: $\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$

مصفوفات التحويلات الهندسية 4×4 :

3D translation:

$$T(dx, dy, dz) = \begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D scaling:

$$S(Sx, Sy, Sz) = \begin{bmatrix} Sx & 0 & 0 & 0 \\ 0 & Sy & 0 & 0 \\ 0 & 0 & Sz & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D rotation:

الدوران في 2D كان حول نقطة أما الدوران في الـ 3D فهو حول محور (axis) وتختلف مصفوفة الدوران وفقاً لمحور الدوران:

1. الدوران حول محور X :

$$R_{x\theta} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. الدوران حول محور Y :

$$R_{y\theta} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

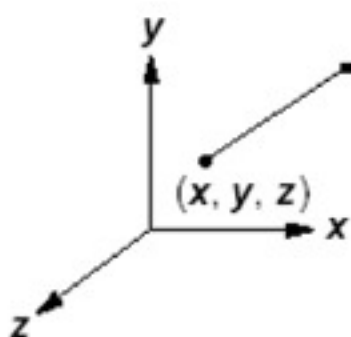
3. الدوران حول محور Z :

$$R_z = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

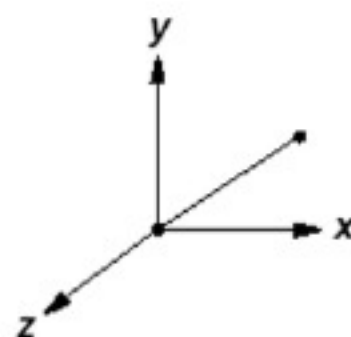
ماذا لو أردنا تدوير جسم حول محور ما عشوائي؟

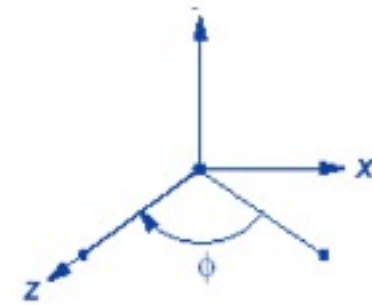
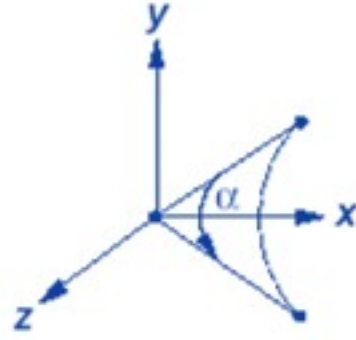
سنجعل هذا المحور ينطبق على المحور z وللقيام بذلك:

■ نحتاج أولاً انسحاب لتطبيق بداية المحور على مبدأ الأحداثيات: $T(-x, -y, -z)$



$$T_{tr} = \begin{bmatrix} 1 & 0 & 0 & -x \\ 0 & 1 & 0 & -y \\ 0 & 0 & 1 & -z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$





■ نحتاج دوران حول x بزاوية α : $Trx(\alpha)$

نضرب بمصفوفة الدوران حول x : $Trx(\alpha)$

■ نطبق دوران حول y بزاوية ϕ :

نضرب بمصفوفة الدوران حول y : $Try(\phi)$

ينطبق الآن محور الدوران على محور z فنطبق الدوران الحقيقي بزاوية θ المطلوبة حول z ولإعادة المحاور نطبق دوران حول y ثم دوران حول x ثم انسحاب $T(x, y, z)$

إذا لدينا 7 تحويلات هندسية أي 7 مصفوفات:

$$Trab = Tr(x, y, z).Trx(-\alpha).Try(-\phi).Trz(\theta).Try(\phi).Trx(\alpha).Tr(-x, -y, -z)$$

Object transformation vs Coordinate transformation

ان التحويلات الهندسية السابقة تنطبق على الغرض (object) وتطبق عن طريق ضرب المصفوفات من يسار النقطة:

$$P' = t2.t1.P$$

ولكن المشكلة في الكمبيوتر غرافيكس أن العرض ليس له صفات فيزيائية بل هو عبارة عن Pixels فعند تطبيق الإزاحة مثلاً ما يتم فعلياً هو مسح الغرض من مكان ورسمه في مكان آخر على بعد صغير جداً وهكذا (التتابع الجاهزة في OpenGL تستطيع رسم 60 صورة في الثانية).

ولكن إذا أردنا التحريك في نفس الصورة فإن التحويلات لا تطبق على العرض وإنما تطبق على محاور الأحداثيات، ففي الانسحاب نحرك مبدأ الأحداثيات، في التكبير نكبر شعاع الواحدة، وفي الدوران ندور المحاور (إذا الأجسام التي سيتم رسمها بعد ذلك سيتم رسمها في جملة الأحداثيات الجديدة).

مثال:

لتطبيق Translate على جسم ما مركزه مبدأ الأحداثيات نقوم بإزاحة جملة الأحداثيات إلى اليسار فيتم رسم الجسم تلقائياً على اليسار، لسنا مضطرين لإزاحة الجسم إلى اليسار.

عندما نطبق التحويلات على الغرض نقوم بـ translate ثم rotate حول النقطة المطلوبة، لكن هنا بعد تطبيق انسحاب سيكون الدوران حول نقطة جديدة بسبب تغير جملة الإحداثيات، لذا ستتراكب التحويلات بطريقة مختلفة ويكون ضرب المصفوفات من اليسار إلى اليمين، أي ستصبح التحويلات معكوسة الترتيب.

لا تقتصر فائدة ال coordinate transformation على نقل الغرض فقط، بل تستطيع أيضاً الانتقال من جملة إحداثيات إلى أخرى

مثلاً

■ نعرف جملة إحداثيات وفي كل جملة لدينا object، هل يمكن حساب إحداثيات الغرض في جملة الإحداثيات الأولى بالنسبة لجملة الإحداثيات الثانية؟ أي نقل الغرض من الجملة الأولى إلى الجملة الثانية؟

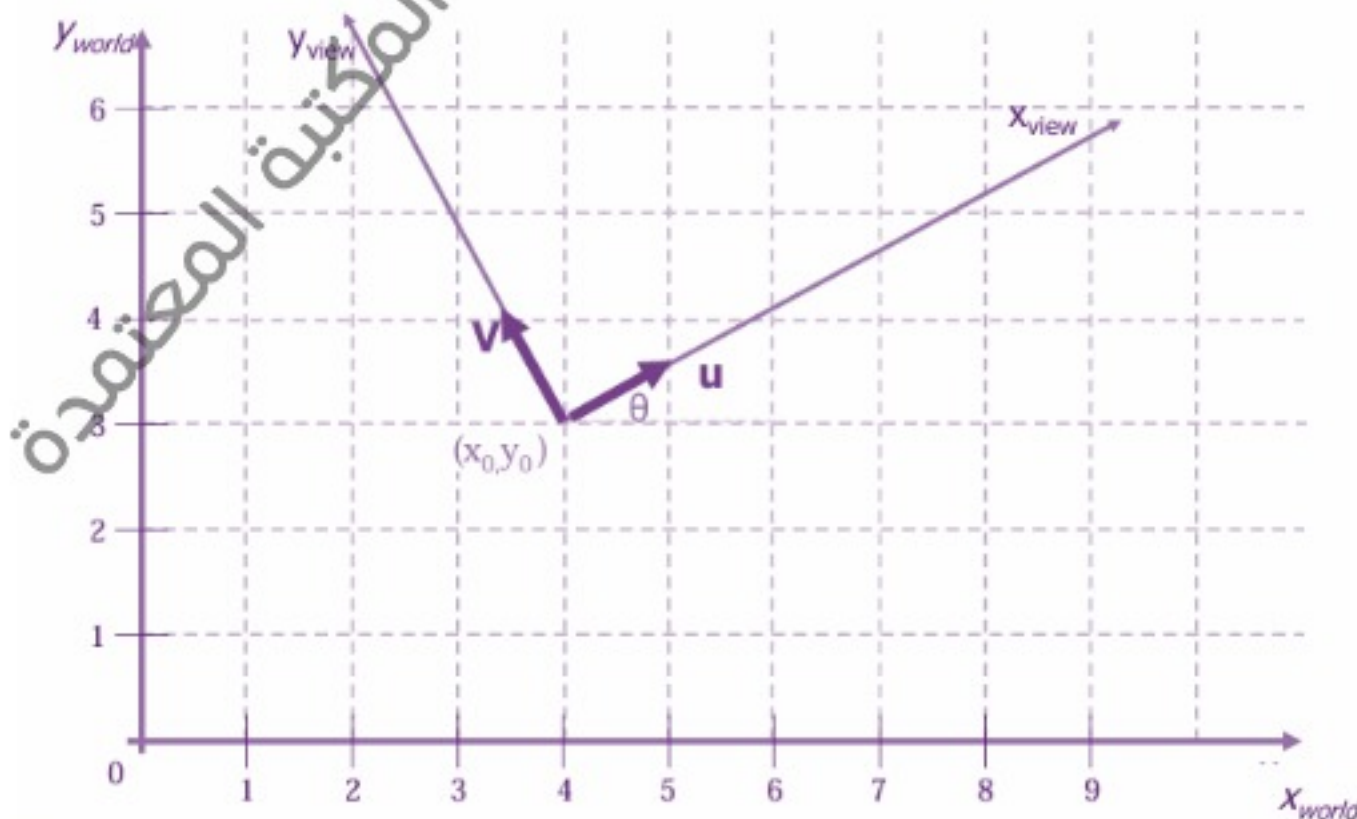
يلاحظ شيئان:

- موقع جملة الاحداثيات الثانية بالنسبة للأولى
- دوران الاحداثيات الثانية بالنسبة للأولى



إذاً لدينا تحويلان: rotate و translate

- الـ translate هو بعد الجملة الثانية عن الأولى
 - الـ rotate هو الزاوية بين محور x في الجملة الأولى ومحور x في الجملة الثانية وهو قياس الزاوية ذاته بين محور y في الأولى ومحور y في الثانية.
- ولكن بدلاً من حساب زاوية الدوران بما أن طول شعاع الواحدة هو 1 فإن x تساوي $\cos\theta$ و y تساوي $\sin\theta$



$$R = \begin{bmatrix} u_x & u_y & 0 \\ v_x & v_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



وفي جملة 3D :

$$R(u, v, w) = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

إذا كان لدينا u و v نحسب w عن طريق جداء الخارجي.

Viewing in 3D

مجموعة الخطوات التي نطبقها للانتقال من الـ 3D Model الذي يمثل الجسم إلى إحداثيات الشاشة للقيام بعملية الرسم بشكل حقيقي يجب معرفة الإحداثيات في الشاشة، فمثلاً لرسم مستقيم على الشاشة يجب معرفة إحداثيات نقطة البداية ونقطة النهاية ولكن بالـ pixel، وإن عملية الـ viewing تحول كل نقطة في جملة إحداثيات 3D لتصبح في جملة إحداثيات الشاشة.

❗ إذا كان الشكل بالأساس ضمن إحداثية ثنائية البعد 2D، هل يمكن رسمه مباشرة على الشاشة؟ لا يمكن لأننا يجب أن نضمن أن تكون الإحداثيات بالـ pixel، كما أن مبدأ إحداثيات الشاشة في الزاوية العليا اليسرى أي علينا الانتقال إلى إحداثيات الشاشة، لذلك نقوم بعملية translate، كما يلزمنا عمليات scale مثلاً كل 1cm يساوي 10 pixel، أي 10 على x و 10 على y (لكي نقلب المحور يكون للأسفل) يلزمنا فعلياً: (translate → scale → translate) إذا كانت جملة الإحداثيات 3D نحول إلى 2D ثم تحول إلى إحداثيات الشاشة طريق عمليات scale و translate.

Viewing in 3D: التحويل من جملة إحداثيات ثلاثية البعد تسمى جملة إحداثيات العالم، (world coordinate) إلى جملة إحداثيات الشاشة (device in coordinate)

سنتعرف على الخطوات التي يتم في الانتقال بين مجموعة من الفضاءات spaces، وصولاً إلى الـ Screen space:

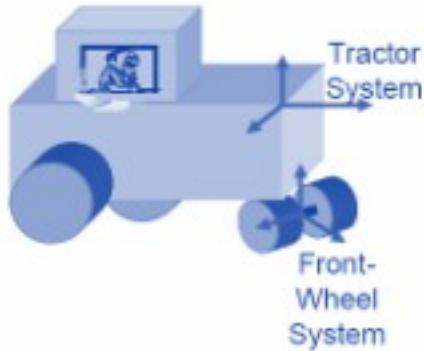


ما دام في قلوبنا أمل سنحقق الحلم سنمضي إلى الأمام
ولن تقف في دروبنا الصعاب لندخل في سباق الحياة ونحقق الفوز
بعزمنا فاليأس والاستسلام ليست من شيمنا

3D Viewing Pipeline

1. Model space

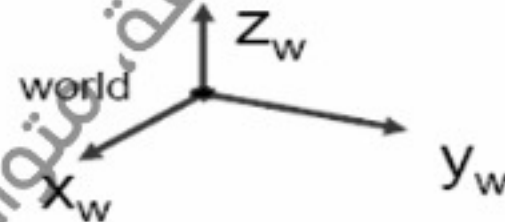
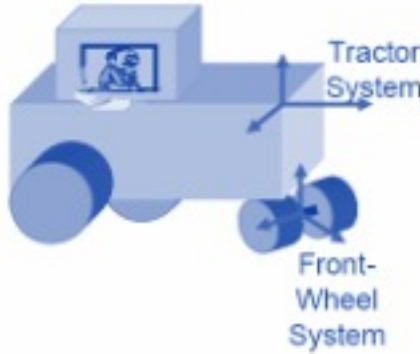
هي جملة إحداثيات الغرض، نفترض جملة إحداثيات نربطها بالـ Model يفضل ان تكون في مركز الـ Model



ولتمثيل الأغراض بالنسبة لبعضها، علينا وضعها في جملة إحداثيات مشتركة: مثلاً لنقول إن الكرسي بعيد عن الطاولة علينا وضعها في جملة إحداثيات واحدة، هذه الجملة تسمى جملة إحداثيات العالم (The world coordinate)

2. World space

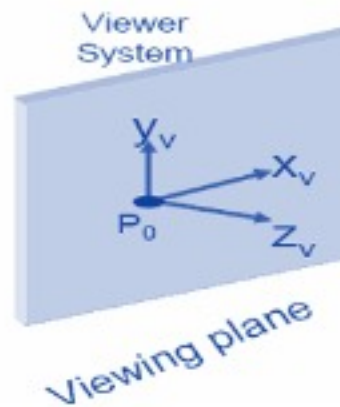
تحدد موقع جملة إحداثيات العالم، ثم تحول كل نقاط الجسم من جملة إحداثيات الجسم إلى جملة إحداثيات العالم.



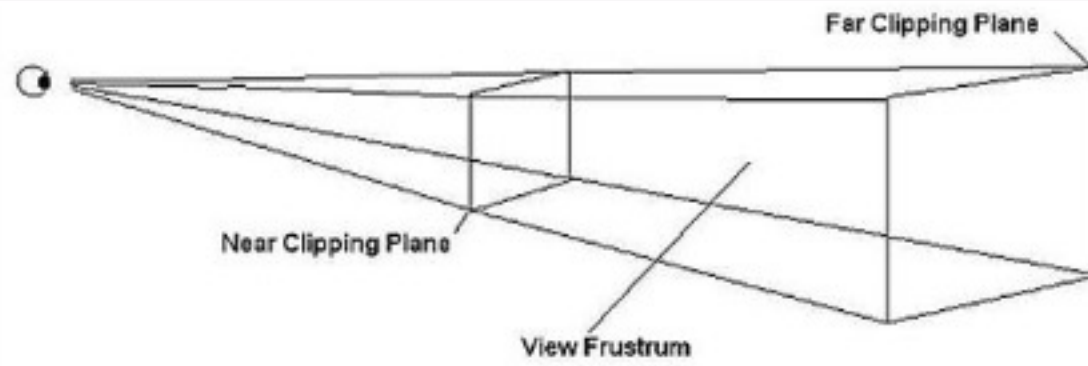
3. view space (or camera space)

إن الناتج النهائي للصورة التي نريد رسمها هو ما تراه الكاميرا، وبالتالي يهملنا موقع الأغراض بالنسبة للكاميرا وإن موقع الجسم بالناتج الناعي متعلق بموقعه بالنسبة للكاميرا، والكاميرا تحاكي عين الإنسان أي تستطيع رؤية الأغراض في مجال الرؤية فقط.

وللتحويل لجملة إحداثيات الكاميرا نضع مبدأ الإحداثيات موقع الكاميرا والمحاور x, y لها علاقة بجهة الكاميرا (ستحدث عن الكاميرا لاحقاً وبالتفصيل).



لكن الكاميرا لا ترى كل المشهد بل جزء منه حسب جهتها نعلم أن مجال الرؤية لدى الإنسان هو مخروط، لكن مجال الرؤية للكاميرا هو جذع هرم وتسمى هذه المنطقة منطقة العرض viewing volume، والأجسام في الـ view volume هي الأجسام التي تراها الكاميرا.



4. Clip space

لكيلا نضطر لرسم أغراض معقدة ولا تراها الكاميرا، نقوم بعملية تسمى clipping. وهي تحديد الأجسام التي تقع داخل الـ view volume والأجسام خارج منطقة العرض، أو التي جزء من داخل المنطقة والجزء الآخر لا تراه الكاميرا، وذلك يخفف استهلاك الأداء، ولكن عملية حساب إذا كان الجسم داخل الهرم أو خارجه هي عملية معقدة رياضياً. ولحل هذه المشكلة ثم تغير خصائص جملة الإحداثيات بحيث يصبح الهرم منتظم لأن خوارزميات حساب التقاطع مع هرم منتظم أسهل. تسمى المنطقة التي حصلنا عليها بعد التعديل بـ (clip space) وهي نفسها منطقة العرض. لكن مع تعديل ليصبح حساب الـ clipping أسهل رياضياً.

الآن أصبح لدينا الأغراض التي نشاهدها فقط مع موقعها الصحيح، إذاً ما المرحلة التالية؟

5. Screen space

عملية الـ projection:

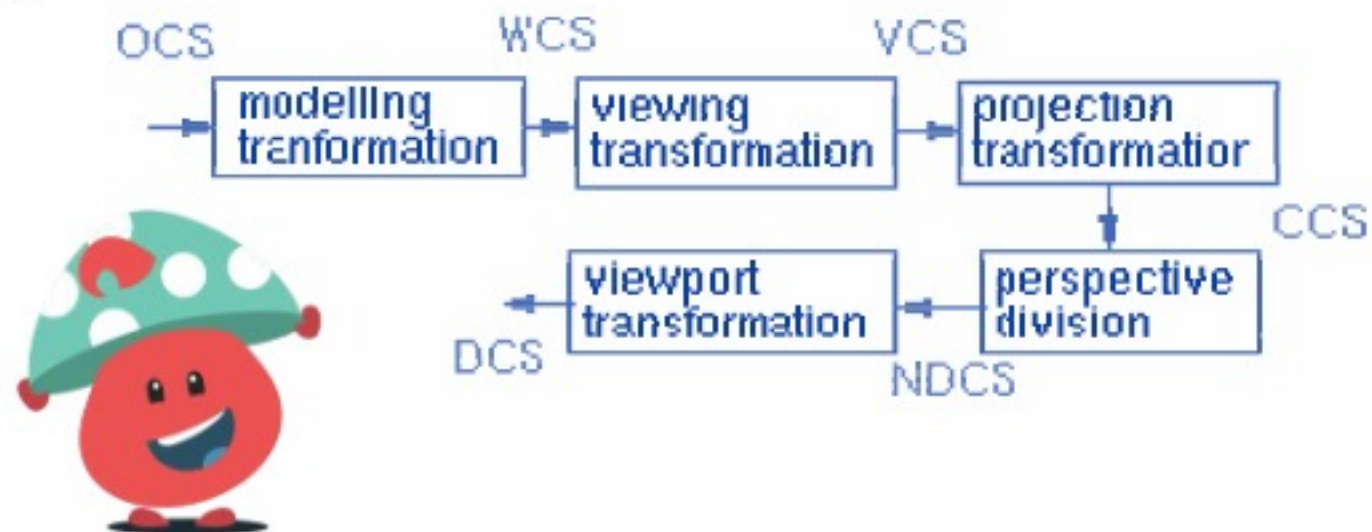
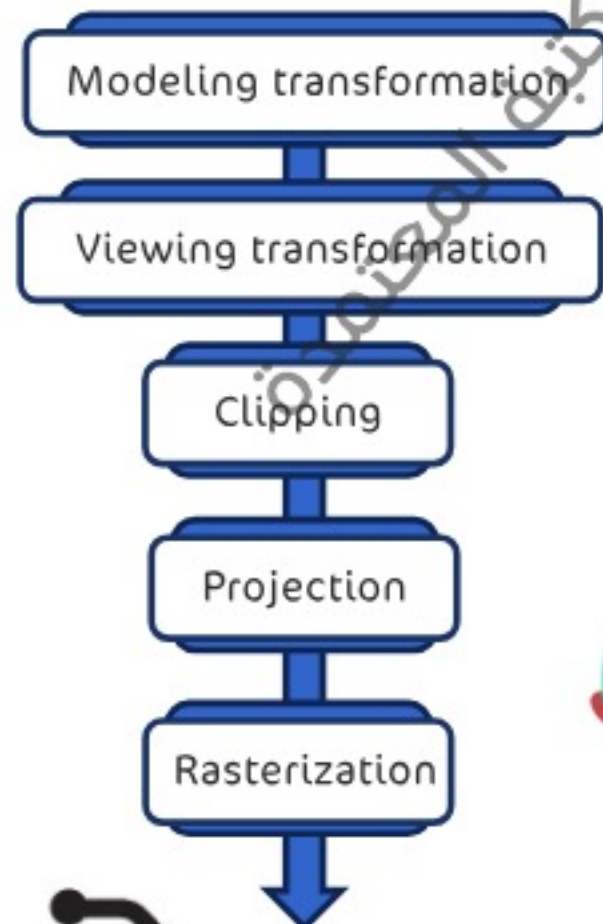
العملية الرياضية التي ستحول الأغراض من 3D إلى 2D، وبالتالي كل الإحداثيات بعد الـ projection ستصبح 2D مع البقاء في جملة إحداثيات الـ clipping.

ملاحظة: عملية الـ projection لا تغير جملة الإحداثيات بل تغير إحداثيات النقاط من 3D إلى 2D مع البقاء في جملة الإحداثيات نفسها.

Mapping:

الخطوة الأخيرة هي عمليات الـ translate والـ scale التي تحدثنا عنها في البداية.

أصبحت النقاط في جملة إحداثيات الشاشة (device coordinate)



ماذا يلزمنا لعملية العرض؟

- تحديد الـ view volume من خلال تحديد موقع الكاميرا وخصائصها.
- تحديد نوع الـ projection وخصائصه وهي عملية معقدة سنتحدث عنها لاحقاً
- تحديد الـ Viewport: يعني تحديد المنطقة ضمن الشاشة التي نريد الرسم عليه، أي عند الرسم تحدد إحداثيات الـ viewport وليس إحداثيات الشاشة، لذلك نحتاج عملية translate للانتقال من إحداثيات الشاشة إلى إحداثيات الـ viewport.

3D pipeline High Level Overview

يوجد ثلاث مراحل أساسية للرسم:

1. Application/Scene

عمليات ليس لها علاقة بالـ Graphics system بل هي دعم مثل:

object visibility check مثلاً إذا أردنا تصميم لعبة فيها لاعب أمامه حائط وخلف الحائط يوجد سيارة ، نظام الغرافيكس يعتبر أنه يمكننا رؤية السيارة لأنها تقع ضمن مجال الرؤية لذا عندما نتأكد أنها على غير مرئية فلا نرسلها لنظام العرض.

مثال آخر: عند دخول اللاعب الى غرفة فإن أي شي خارج الغرفة هو غير مرئي

Select Level of Detail (LCD): تصميم الشكل بأكثر من دقة واختيار الدقة المناسبة، مثلاً دقة السيارة في مكان بعيد تختلف عن دقتها في مكان قريب، لذا ترسم السيارة بأكثر من دقة ونختار الشكل المناسب حسب بعد و قرب السيارة عن الكاميرا.

هذه العمليات يقوم بها الـ developer قبل نظام الإظهار

2. Geometry

تتضمن كل العمليات والتحويلات التي تحدثنا عنها سابقاً

3. Rendering / Rasterization

وهي عمليات الإظهار الأخيرة مثل:

- Shading (تتعلق بالظلال والإضاءة)
- Texturing
- Antialiasing (التكسر)
- Alpha Translucency tests
- Depth Buffering



تقنية تستخدم لرسم الأشكال بطريقة أكثر واقعية، مثلاً إذا رسمنا مكعب ثم رسمنا مكعب آخر أكبر أمامه نظام الغرافيكس لا يمكنه تحديد أن المكعب الكبير سيغطي الصغير لذلك تقنية الـ Depth Buffering تساعد في حل هذه المشكلة.

■ Display

نعلم أن التحويل من جملة إحداثيات الـ Model إلى جملة إحداثيات العالم ثم من العالم إلى الكاميرا هو جداء مصفوفات، لذا يمكننا الحصول على مصفوفة واحدة ناتجة عن جداء المصفوفتين للانتقال مباشرة إلى جملة إحداثيات الشاشة دون المرور بالعالم، وتسمى Model-view matrix.

أن الكاميرا لا يمكن تحريكها أو تغيير مبدئها لأنها فيزيائياً ملتصقة بالشاشة، لذا عند تقريب الكاميرا أو إبعادها ما يتحرك فعلياً هو إحداثيات العالم وليس الكاميرا، وبالتالي لا يوجد توابع التحريك للكاميرا. في OpenGL يوجد تابع يسمى GLU logout وهو تابع مساعد يأخذ خصائص ويستخدم لتحريك الكاميرا، لكنه ضمناً يحرك العالم.

The End

نحن نعيش لكي نرسم ابتسامة .. ونمسح دموعاً ..
ونخفف ألماً .. ولأن الغد ينتظرنا .. والماضي قد رحل
وقد تواعدنا مع أفق الفجر الجديد.



المكتبة المعتمدة